University of Bedfordshire

Elevator Controllers

3/11/2026

Name: Nimesh Karki
UNIVERSITY ID:2439463

Table Of Contents

1.Introduction 5

1.1 Problem Statement 5

1.2 Main Challenges 5

1.3 Proposed Solution 5

2. Aim 5

3. Objectives 6

4. System Analysis 6

4.1 Functional Requirements 6

4.2 Non-Functional Requirements 7

5. System Architecture 8

5.1 User Interface 8

5.2 Elevator Control Logic 8

5.3 Database Management 8

5.4 Background Processing 8

6. System Design 9

6.1 Use Case Diagram. 9

6.2 Class Diagram 10

6.3 Activity Diagram 11

6.4 Entity Relationship Diagram 12

8. Object-Oriented Programming 13

8.1 Encapsulation 13

8.2 Inheritance 13

8.3 Polymorphism 13

8.4 Abstraction 13

8.5 Event-Driven Programming 14

8.6 Resource Management 14

9. BackgroundWorker 14

10. Design Patterns 14

10.1 Observer 14
10.2 State-Based Logic 14
10.3 Background Processing 14
10.4 Separation of Responsibilities 15
11. Testing 16
12. Self-Assessment 25
14. Limitations 26
15. Future Enhancements 26
16. Conclusion 26

Table of Figure:

Figure 1Use Case Diagram **8**
Figure 2Class Diagram **9**
Figure 3 Activity Diagram **10**
Figure 4 ER- Diagram **11**

Table of Table:

Table 1Functional Requirements **5**
Table 2Non-Functional Requirements **6**
Table 3 Self-Assessment **24**

1.Introduction

This report describes a C# Windows Forms application that can control two elevators in a two-story building. The system is based on an external caller dispatcher that sends an external call to the closest lift.

And lifts move in a state based logic. Every action gets logged into SQLite database and with the use of background workers to stop the GUI from freezing. This system's design, implementation, and testing have been covered in the report.

1.1 Problem Statement

There is an elevator control system installed in a two-floor office building. The system should receive and respond to calls from the floors and requests from inside each car, cause smooth movement of the cars and log all activities.

1.2 Main Challenges

Coordinating both lifts so the closer one responds to the call was the main challenge. The other challenges were keeping the displays and car positions in sync with the actual movement, freezing of the window while we store the log records and database error handling without the program crashing.

1.3 Proposed Solution

This software was coded in C# using a Windows Forms environment. Every lift is controlled by a state-based object which chooses the lift for each outside call or request. Furthermore, every change is captured in a SQLite database through a background worker.

2. Aim

The goal of this assignment is to learn to solve a real-life problem using C#. The project simulates a 2 lift building and is an object-oriented design with event driven programming and data base SQ-Llite (Db browser).

3. Objectives

- Create a visual representation featuring buttons for calling, panels for lifts, displays for floor status, and views for logs.
- The control logic should be implemented that will move the correct lift for each request and update every display.
- Store database operation records with timestamps and display them in the interface.
- Using timer and event handling to animate lift movement.
- Expand the system to include two lifts that use the State, Observer pattern and send outside calls to the nearest lift.

4. System Analysis

4.1 Functional Requirements

ID	Requirement Description
FR1	Pressing an external call button dispatches the nearest available lift to that floor.
FR2	Pressing a floor button inside a lift car directs only that specific car to the selected floor.
FR3	The system displays each lift's current floor, direction of travel, and door status.
FR4	Every lift state change is recorded in the database, including the lift name, floor, action, and timestamp.
FR5	The recorded operation logs can be retrieved and viewed within the GUI.

Table 1Functional Requirements

4.2 Non-Functional Requirements

ID	Requirement Description
NFR1	**Portability:** A relative path is used for the database file, allowing the application to run from any directory or machine.
NFR2	**Responsiveness:** Database operations run asynchronously in the background to keep the user interface responsive.
NFR3	**Robustness:** Exceptions are caught and reported to the user via a message box to prevent the application from crashing.
NFR4	**Usability:** The elevator animation is smooth and updates consistently every 50 milliseconds.

Table 2Non-Functional Requirements

5. System Architecture

5.1 User Interface

Form1 serves as the primary window. A label indicates which lift is located at each floor, and there is a call button for each lift. Each lift has a panel that has a floor display, two floor buttons, Open and Close buttons and door status label. The custom CarPanel control is responsible for drawing the car with some steel doors that slide open and closed. The stored records are loaded into a grid via the "Show log" button.

5.2 Elevator Control Logic

The ElevatorContext object of each lift maintains the current floor, targeting floor, exact position and a queue of pending stops. Both lift move due to the dispatcher tick 50ms timer. If the door to a lift takes 40 ticks (2 seconds) to open and close, then it will be open for 80 ticks (4 seconds) when moving from one floor to another. Dispatcher. Give checks when a lift is on its way to the floor first.

5.3 Database Management

ElevatorLog employs System.Data.SQLite along with SQLite. The database file elevator.db is located in the Data folder of the application startup path. The constructor will create the Log table if it doesn’t exist already with columns Id, Elevator, Floor, Action and LoggedAt. The Add() method utilizes a SQLiteDataAdapter and a SQLiteCommandBuilder to incorporate a DataTable row and invokes Update(). Load() fills a DataTable with new records.

5.4 Background Processing

The LogService class works with two BackgroundWorker objects that save queued log entries and load records to the grid. The queue is synchronously added to by the interface thread. The form receives errors from either worker via the Failed event.

6. System Design

6.1 Use Case Diagram.

The diagram requires this as a reference for his work. The passenger may call the lift, select the destination, open or close the doors, and see the log. The system dependencies, for example, assigning the nearest lift when calling a lift and it moving automatically logs the operation, is shown via an <<include>> relationship.

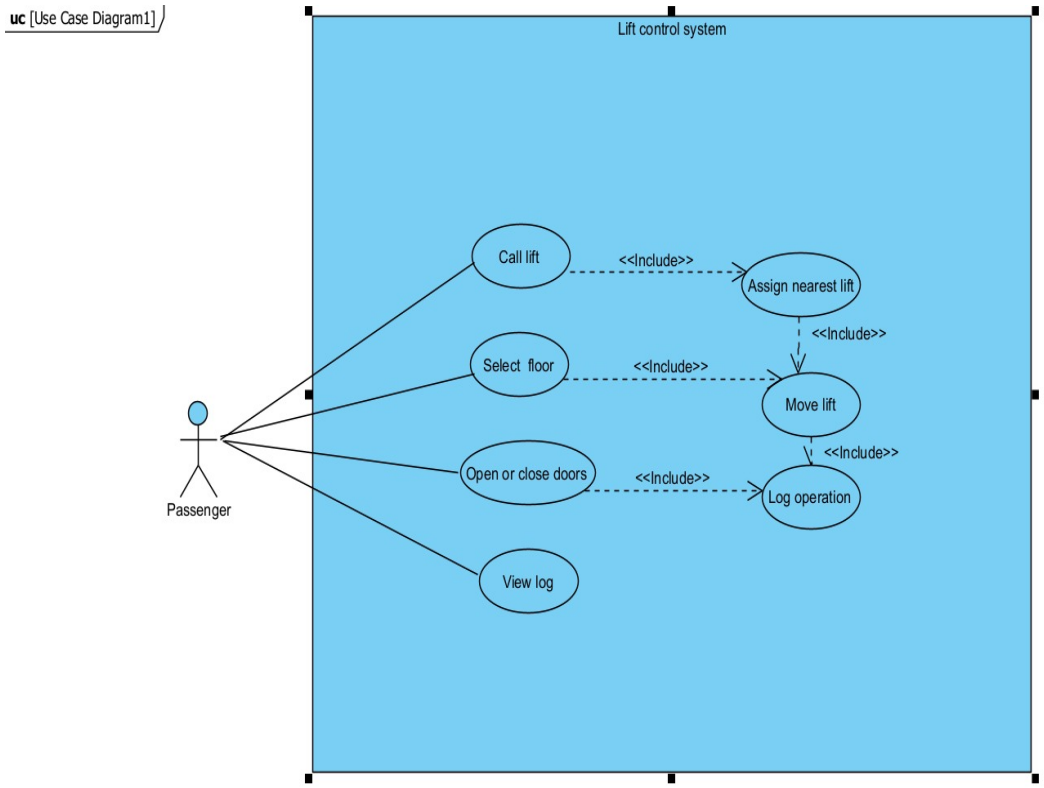


Figure 1Use Case Diagram

6.2 Class Diagram

This structure of a System with State and Observer Patterns is shown through the diagram. The responsibility of the ElevatorContext is ensuring that the lift functions properly and is in the correct state. The object called (Idle, Moving, DoorsOpen) informs the Form1 and LogObserver of changes, which updates the GUI which is perceived with the CarPanel and logs it in the database helped by LogService. The elevators are managed by the dispatcher.

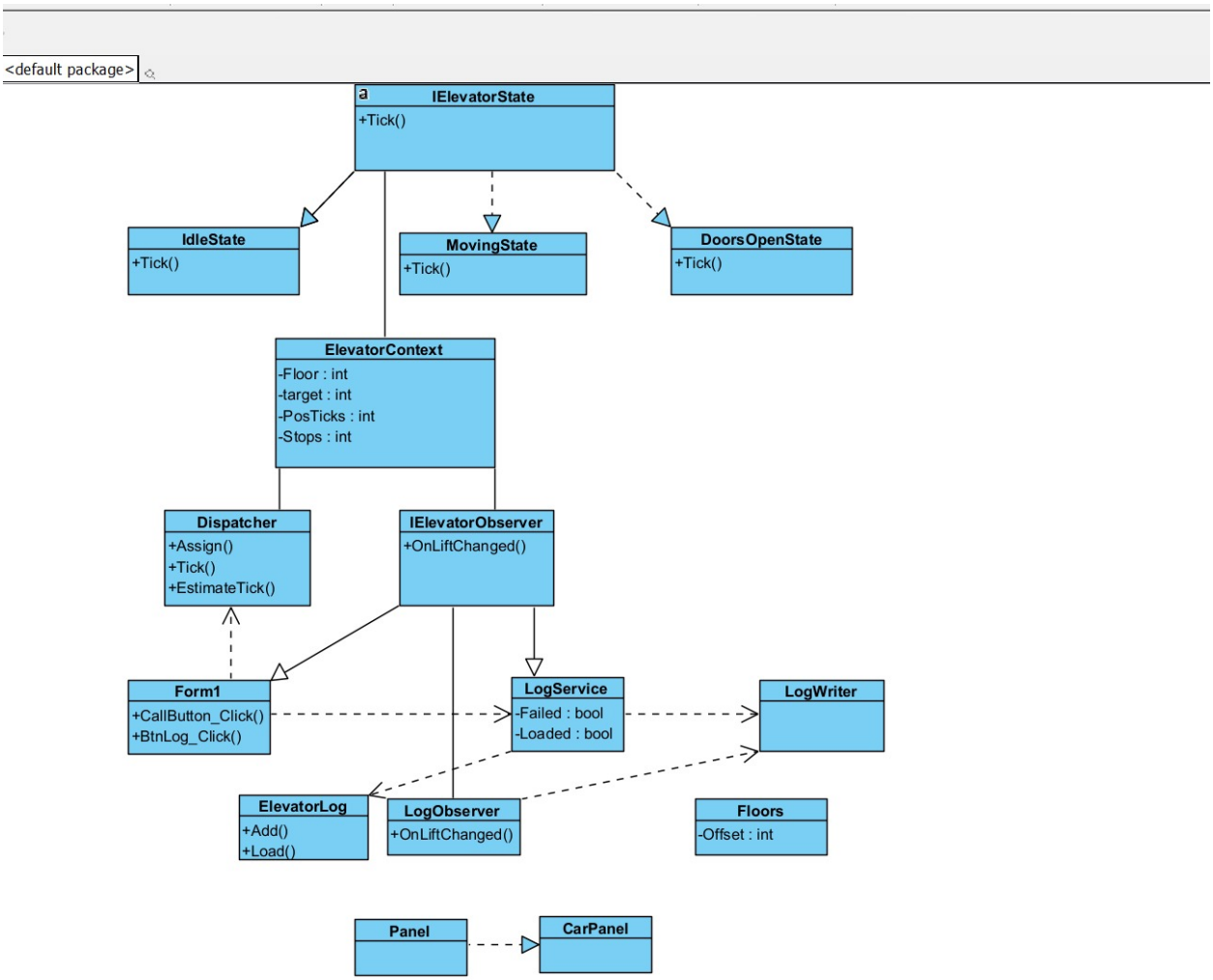


Figure 2Class Diagram

6.3 Activity Diagram

Not included here, as it requires a diagram. Describe the steps for one request in words if your tutor asks for it: button press, dispatcher choice, timer ticks, arrival, doors open, doors close.

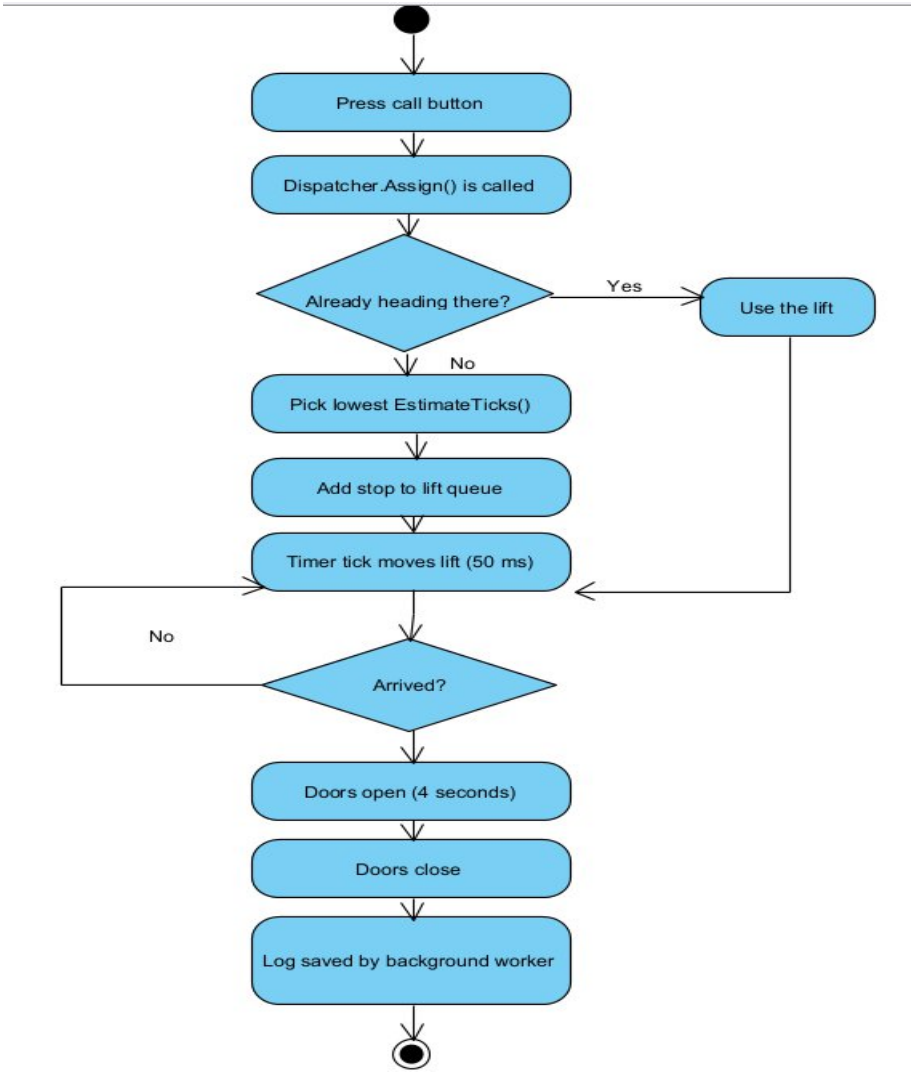


Figure 3 Activity Diagram

6.4 Entity Relationship Diagram

If it does not send it to the lift with the shortest estimation from EstimateTicks. When a request is made from inside a car, it is sent to that lift only through RequestMove.ER diagram of the database for the lift The Log table is the table that is used to log each operation of the lift. It has columns for Id (primary key), name of the Elevator, the Floor that the operation occurs on, and when it is LoggedAt. The only table used is the log table as the states of the elevators is stored in memory by the ElevatorContext object so no other tables are used.

6.4 ER-Diagran

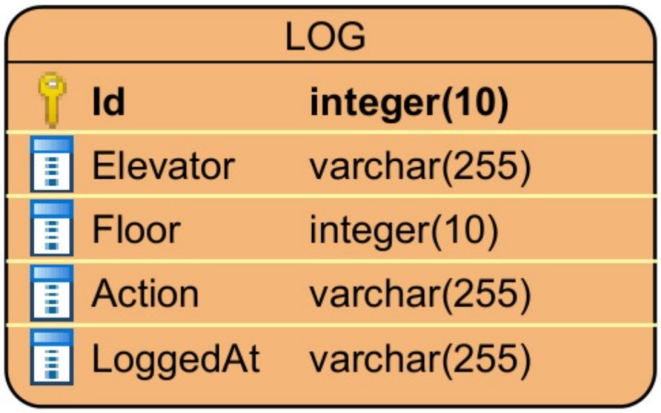


Figure 4 ER- Diagram

7. Technologies and Implementation

- Language: C#
- Framework: Windows Forms, using ApplicationConfiguration.Initialize(), so .NET 6 or later
- IDE: Visual Studio
- Database: SQLite through the System.Data.SQLite package.
- Database file: Data\elevator.db, created on first run
- Timer interval: 50 ms
- Floor numbering:The timer interval duration is set at 50 ms; floor coding: as the code doesn't store the floor numbers as 1 and 2 but as 0 and 1 (Numbers.Offset is 1); this should be stated here, and test cases should use these numbers, or change Offset to 0 meaning floor coding would match the brief.

8. Object-Oriented Programming

8.1 Encapsulation

ElevatorContext. Stops is a private set queue that is only modified through RequestMove. LogService has the queue private lock-protected. Floor, Target, and PosTicks still have public setters, so encapsulation is partial. Make sure to mention this during your assessment.

8.2 Inheritance

CarPanel inherits from Panel and adds custom painting. Form1 inherits from Form.

8.3 Polymorphism

CarPanel is a derived class of Panel, with custom painting. Form1 is a derived class of Form.

8.4 Abstraction

Lift states are encapsulated in IElevatorState; and who receives the lift updates is encapsulated in IElevatorObserver.

8.5 Event-Driven Programming

Button clicks are managed by CallButton_Click, PanelButton_Click, OpenButton_Click, CloseButton_Click, and BtnLog_Click. The timer's Tick event controls movement. LogService triggers Loaded and Failed events.

8.6 Resource Management

The blocks dispose the connection, adapter, and command builder for SQLite. The form1.Dispose disposes the designer components.

9. BackgroundWorker

LogService has 2 workers. The saver pulls entries from the queue one by one and adds them with ElevatorLog. Add. When the saver completes, it queries for more, and if it finds any, it starts pulling again. The loader calls ElevatorLog. Load and returns that table as its result, which is routed to the grid on the interface thread, thereby getting the database activity off of the interface thread.

10. Design Patterns

10.1 Observer

It notifies each time. There are two observers attached, a LogObserver that logs to the log, and a Form1 that updates the lights and displays.

10.2 State-Based Logic

State-Based LogicIdleState, MovingState, and DoorsOpenState are the different states of the elevator. All the states implement the IElevatorState interface. SetState transitions the state and sets the observer.

10.3 Background Processing

All database-related actions occur on a worker thread in the background so the application remains smooth and does not hang when logging elevator events. As discussed in Section 9, the LogService passes off all database reads and writes to a worker thread, effectively decoupling the GUI from the hit against the database.

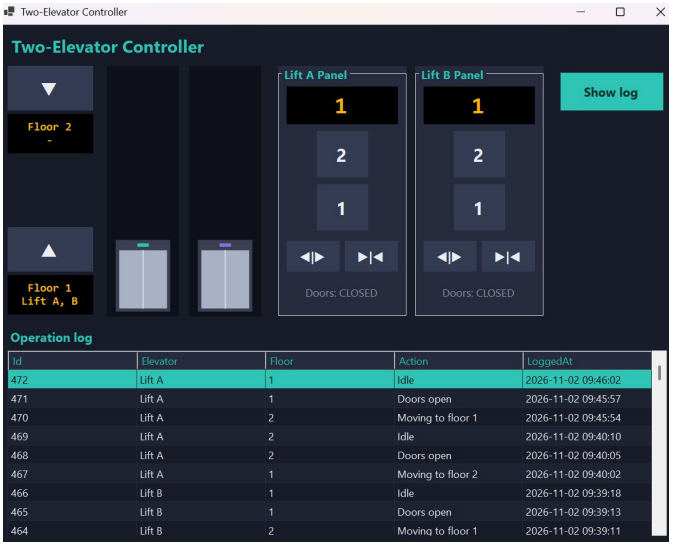
10.4 Separation of Responsibilities

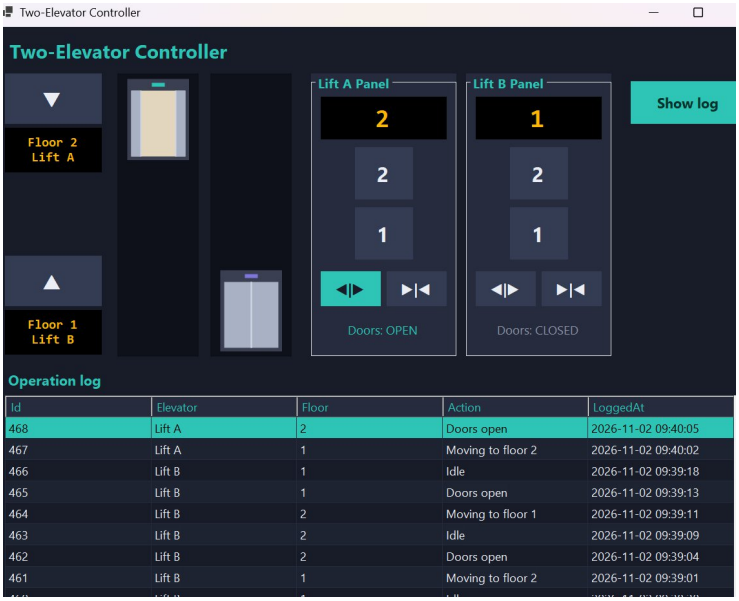
Form1 is only the GUI. The lift logic is contained within the class ElevatorContext, the classes that perform the functions listed in the requirements, and the state classes. The storage of data is the responsibility of the LogService and ElevatorLog classes. The LogObserver class feeds lift events to the logger via the LogWriter delegate, so it is not coupled directly to LogService.

11. Testing

T1: Calling a lift from Floor 2 to Floor 1

Expected: One lift travels to Floor 2 and the doors open. The other stays at Floor 1.

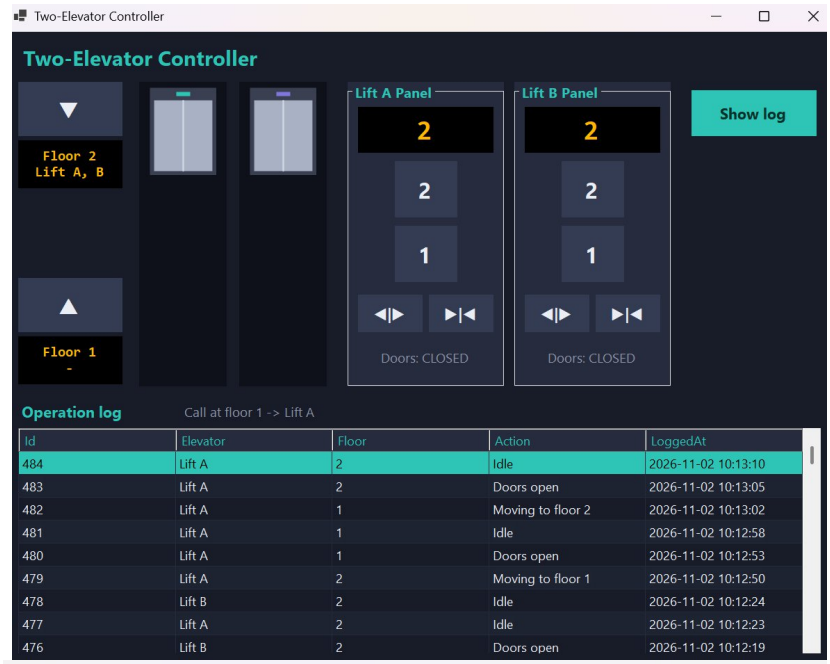


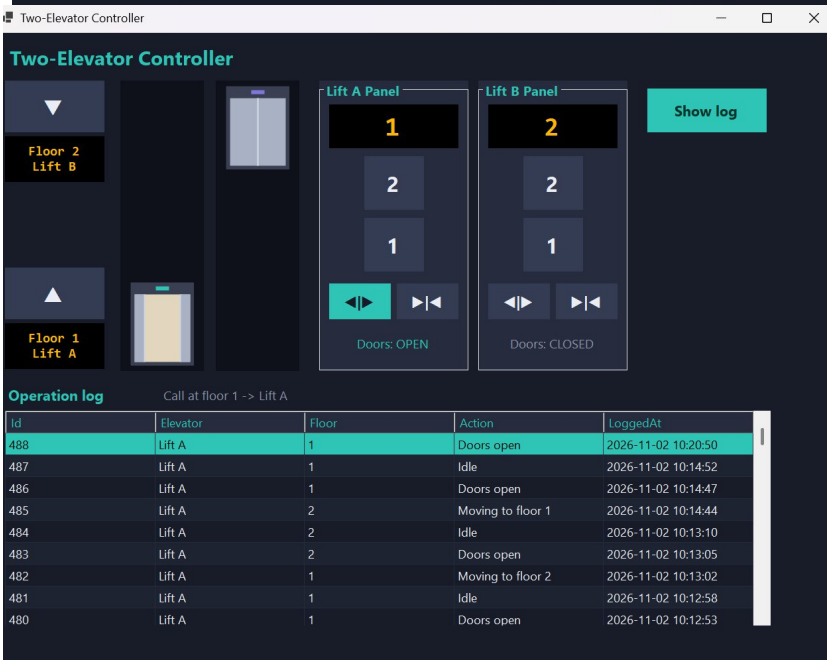


Result: Pass. Lift A answered the call, and Lift B did not move.

T2: Call a lift from Floor 1

Expected: The nearest lift travels to Floor 1 and the doors open.



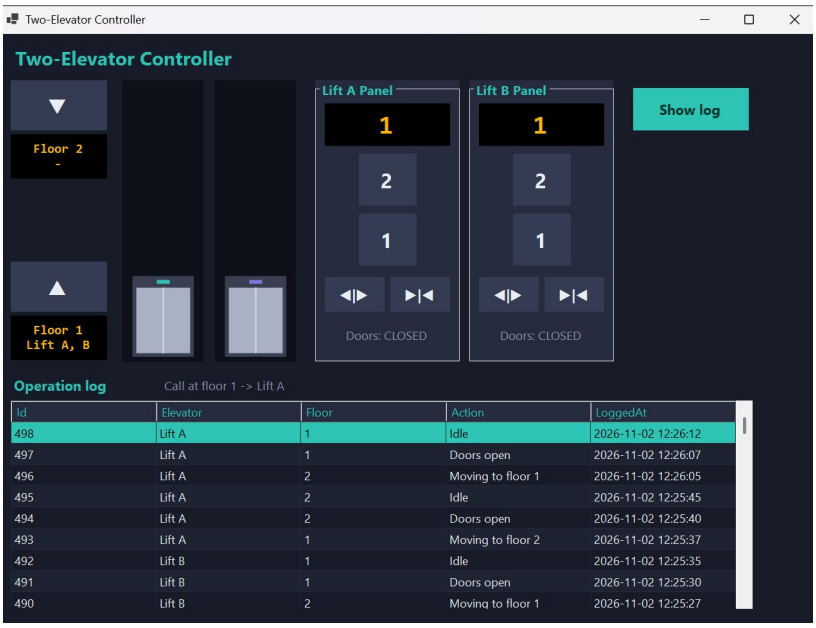


Result: Pass. [Lift A] was the nearest lift, travelled to Floor 1 and opened its doors.

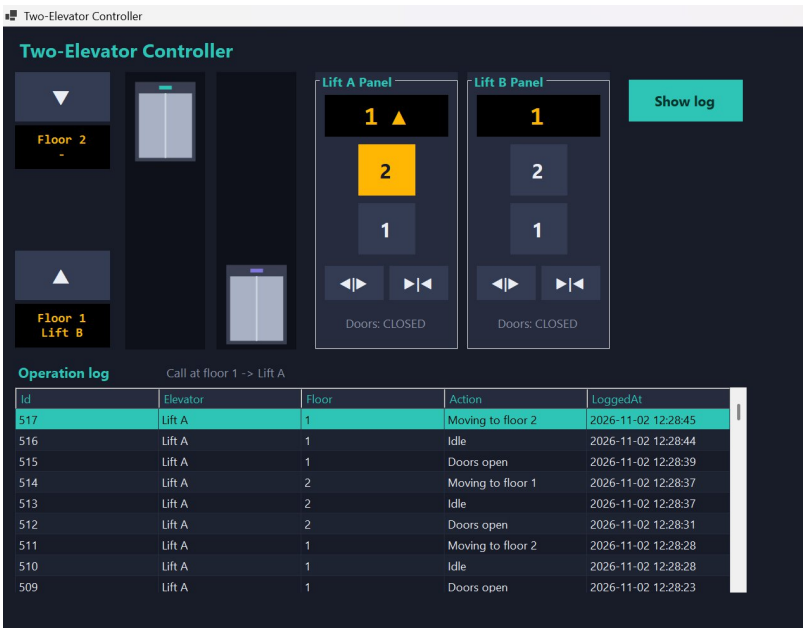
T3: Floor button inside Lift A

Expected: Only Lift A moves to the selected floor. Lift B stays still.

Before: Both lifts at Floor 1.



After: Lift A at Floor 2, Lift B at Floor 1.

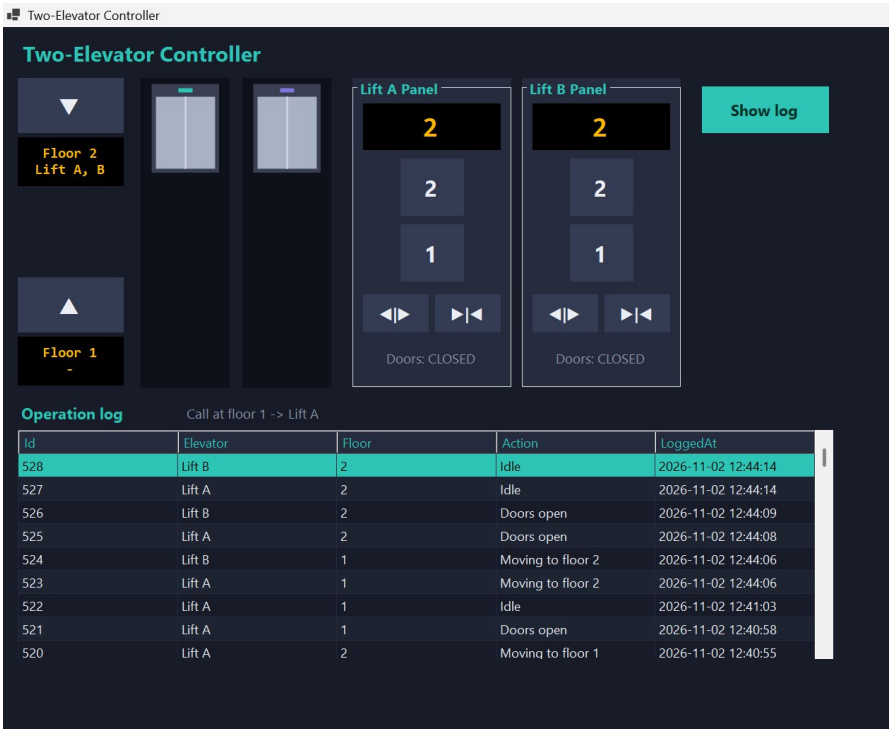


Result: Pass. Lift A moved to Floor 2 and Lift B stayed at Floor 1.

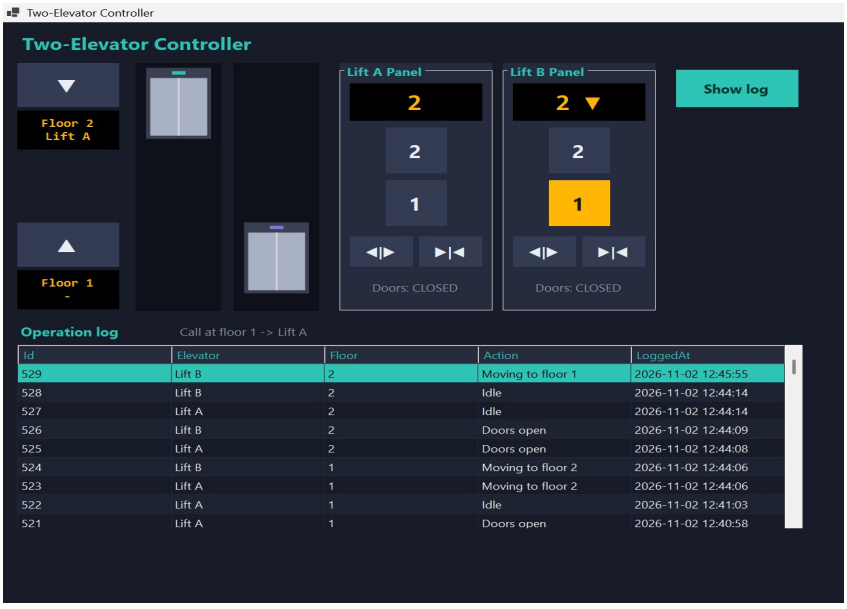
T4: Floor button inside Lift B

Expected: Only Lift B moves to the selected floor. Lift A stays still.

Before: Both lifts at Floor 2.



After: Lift B at Floor 1, Lift A at Floor 2.

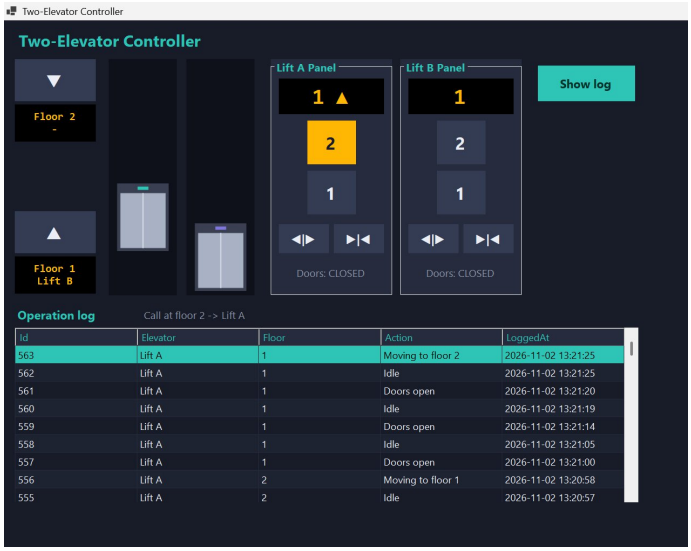


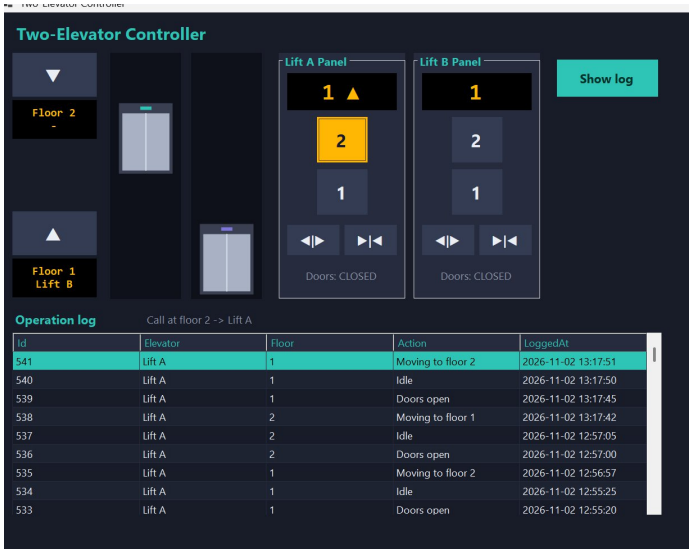
Result: Pass. Lift B moved to Floor 1 and Lift A stayed at Floor 2.

T5: Display updates during movement

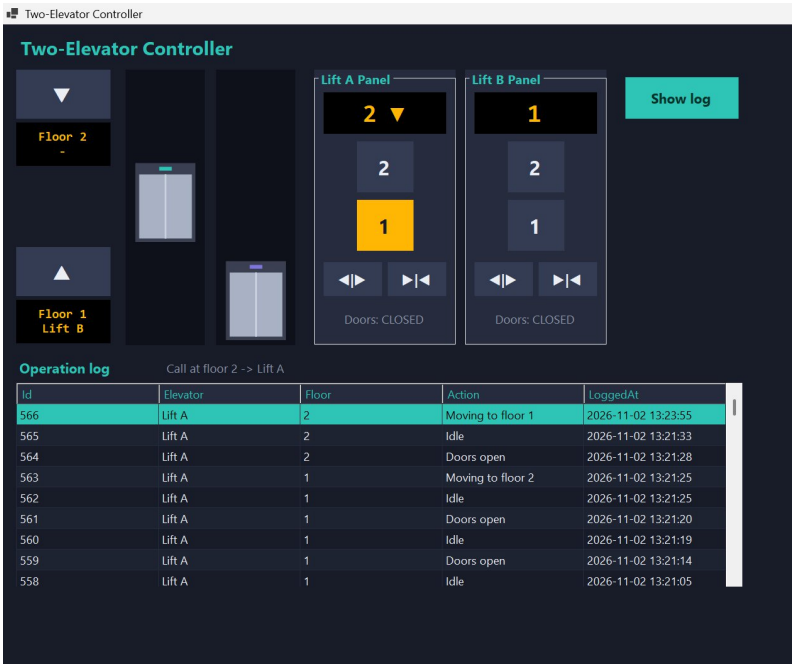
Expected: The floor display, direction and door status change in step with the movement.

Before: Lift idle at Floor 1.





After: Screenshot taken mid-travel, with the display and the car between floors.

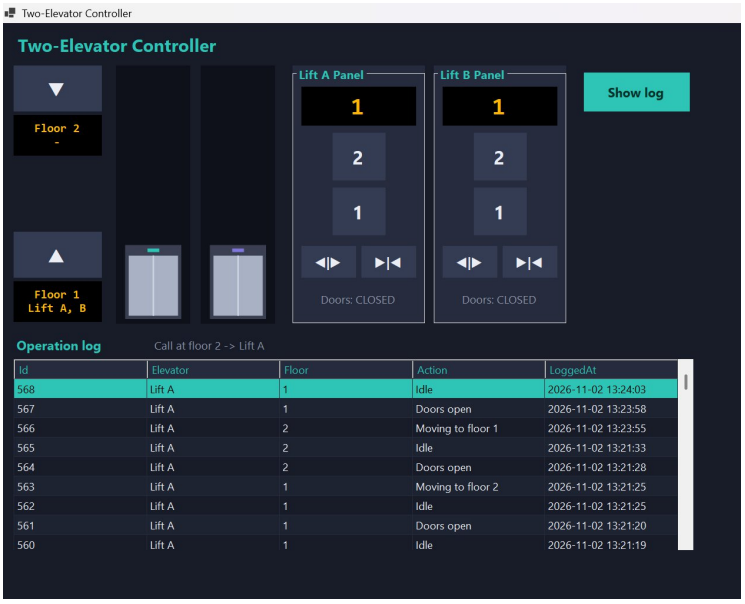


Result: Pass. The floor display, direction and door status changed in step with the car's movement.

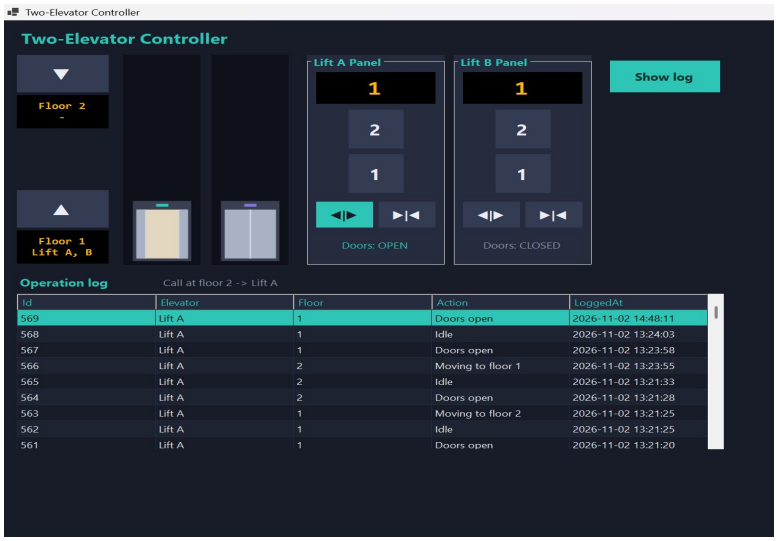
T6: Open button

Expected: The doors open and the status label shows “Doors: OPEN”.

Before: Doors closed.



After: Doors open.

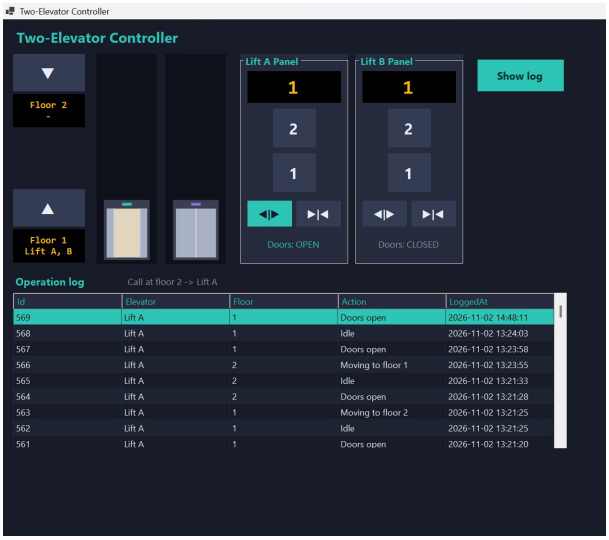


Result: Pass. The doors slid open and the label showed “Doors: OPEN”.

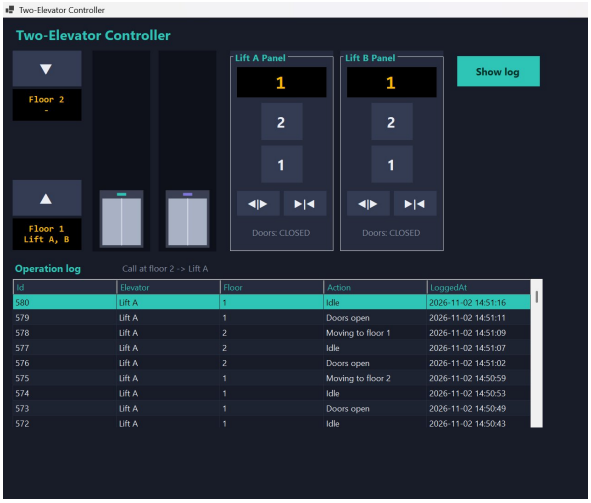
T7: Close button

Expected: The doors close and the status label shows “Doors: CLOSED”.

Before: Doors open.



After: Doors closed.

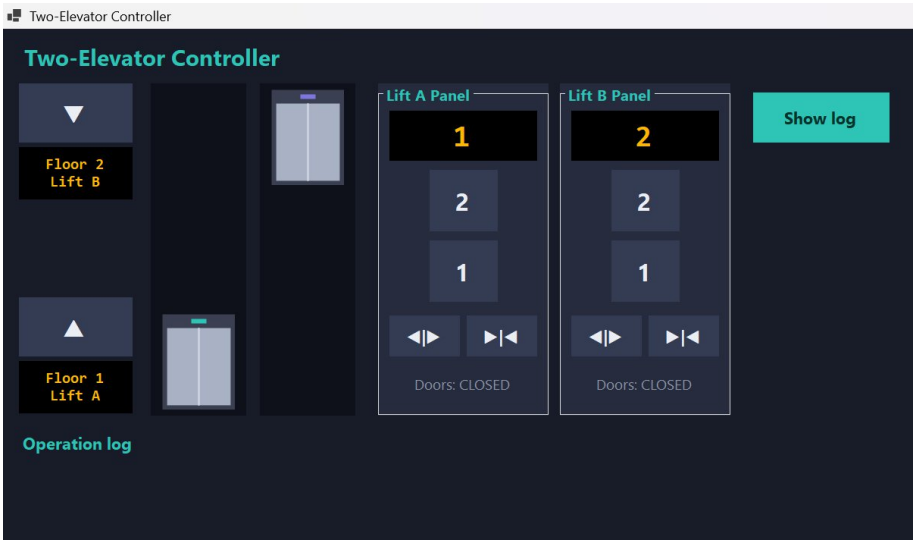


Result: Pass. The doors slid closed and the label showed “Doors: CLOSED”.

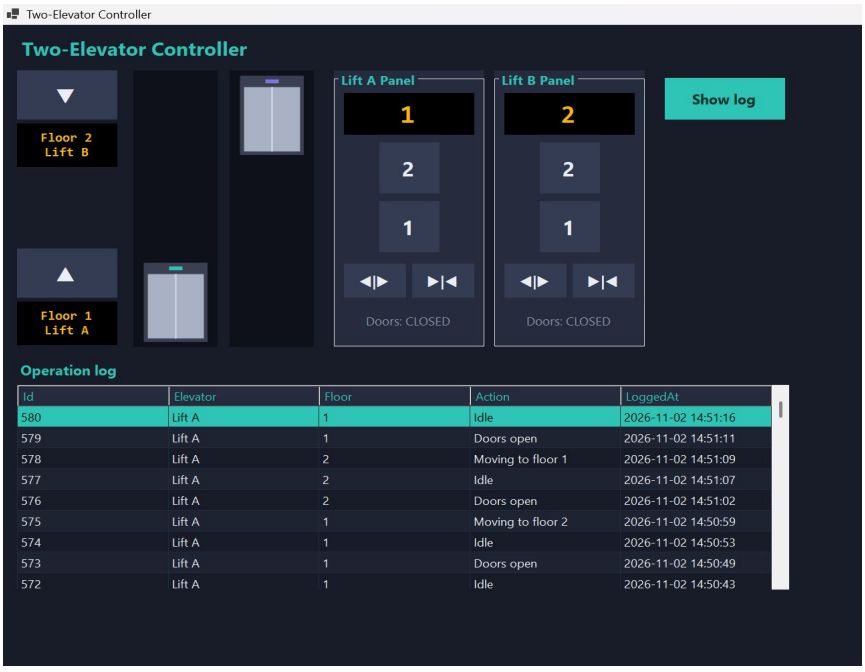
T8: Show log

Expected: The grid lists the records, newest first, with lift name, floor, action and time.

Before: Window before clicking Show log.



After: The log grid filled with rows.

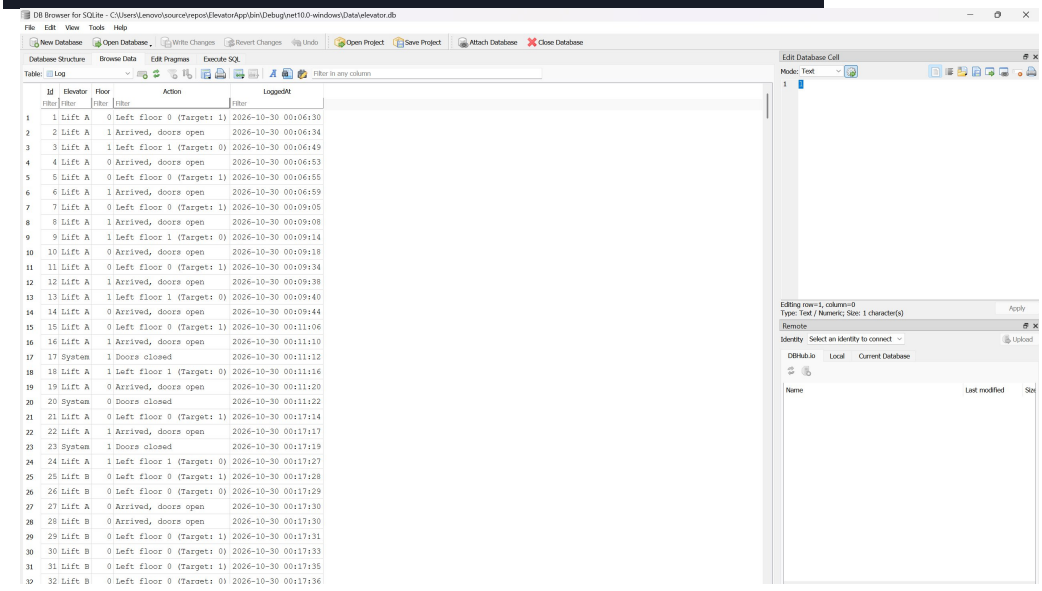


Result: Pass. The grid showed the records with the newest first, each with lift name, floor, action and time.

T9: Database storage checked

Expected: Every lift operation is saved in the SQLite database (Data\elevator.db) and is shown in the Operation log.





Result: Pass. The database file was created automatically in the Data folder, and the Log table held the same records as the grid, with the Id, Elevator, Floor, Action and LoggedAt columns filled in.

12. Self-Assessment

Task 1	Complete GUI for Task 1	10	Yes	T1	
	Event handler skeleton implemented for all GUI buttons	10	Yes	T1–T3	
Task 2	All event handlers function as specified	10	Yes	T2–T4	
Task 3	Database designed and successfully connected	5	Yes	T8, T9	
	Log information retrieved from DB and displayed in GUI	5	Yes	T8	
	Log information stored in DB	5	Yes	T9	
	Disconnected model used (via Data Adapter.Update() instead of Execute-Non-Query())	5	Yes	T9	
	Relative paths used; logical errors eliminated; exceptions handled with try-catch	5	Yes	T6, T7	
	No duplication in DB-related event handlers	5	Yes	T8, T9	
Task 4	Events described in Task 2 animated using delegation and timer	10	Yes	T5	
Task 5	Application efficiency improved by separating GUI and DB operations using Background-Worker	5	Yes	T8, T9	
	System extended to support two elevators; State and Observer Patterns applied to manage elevator logic	15	Yes	T1–T4	
Task 6	Testing report includes all expected content	10	Yes	Section 11	
Total		**100**			

Table 3 Self-Assessment

13. Advantages

- Each lift is independent, so one lift's state does not affect the other's logic.
- The State and Observer patterns separate lift behaviour from display and logging.
- Database work runs in the background, keeping the interface responsive.
- The database path is relative, so the application is portable.

14. Limitations

- Floor, Target and PosTicks have public setters, so the lift state can be changed from outside the class.
- If the database fails to start, the logging observer is not attached. In that case, lift movement still works but nothing is logged. A message box shows the error once at startup.
- Errors from saving or loading are reported through the Failed event and shown in a message box. ElevatorLog.Add() has no try-catch of its own.

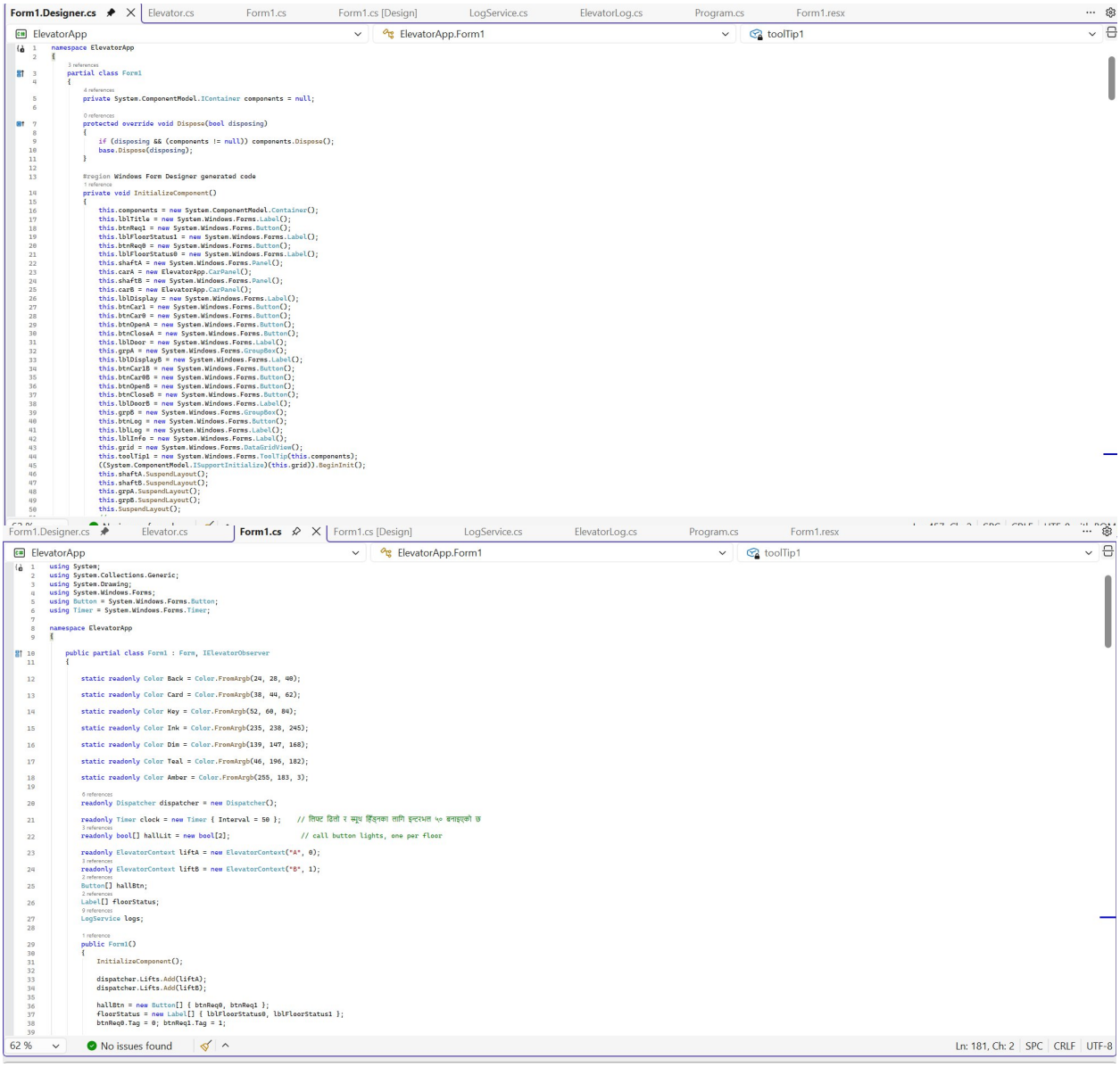
15. Future Enhancements

- Add more floors, with the number of floors fixed in one place.
- Record the position of the lift in the database (to be restored on restart).
- Add a button to clear the log.
- Read Only Floor, Target and PosTicks from outside the class.
- Include a test project and automated unit tests for Dispatcher and the states.

16. Conclusion

The two-lift elevator in this project has developed the C# Windows Forms. The system efficiently manages lift behaviour and UI updates through the implementation of State and Observer patterns. All 9 tests (T1–T9) passed successfully: movement, doors, and database logging. The most valuable lesson learned was integrating object-oriented design with asynchronous programming (BackgroundWorker) for building a responsive application and dependable.

17. Appendices



```
Form1.Designer.cs | Elevator.cs | Form1.cs | Form1.cs [Design] | LogService.cs | ElevatorLog.cs | Program.cs | Form1.resx
ElevatorApp | ElevatorApp.Form1 | toolTip1

namespace ElevatorApp
{
    3 references
    partial class Form1
    {
        4 references
        private System.ComponentModel.IContainer components = null;

        0 references
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null)) components.Dispose();
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code
        1 reference
        private void InitializeComponent()
        {
            this.components = new System.ComponentModel.Container();
            this.lblTitle = new System.Windows.Forms.Label();
            this.btnReq1 = new System.Windows.Forms.Button();
            this.lblFloorStatus1 = new System.Windows.Forms.Label();
            this.btnReq0 = new System.Windows.Forms.Button();
            this.lblFloorStatus0 = new System.Windows.Forms.Label();
            this.shaftA = new System.Windows.Forms.Panel();
            this.carA = new ElevatorApp.CarPanel();
            this.shaftB = new System.Windows.Forms.Panel();
            this.carB = new ElevatorApp.CarPanel();
            this.lblDisplay = new System.Windows.Forms.Label();
            this.btnCar1 = new System.Windows.Forms.Button();
            this.btnCar0 = new System.Windows.Forms.Button();
            this.btnOpenA = new System.Windows.Forms.Button();
            this.btnCloseA = new System.Windows.Forms.Button();
            this.lblDoor = new System.Windows.Forms.Label();
            this.grpA = new System.Windows.Forms.GroupBox();
            this.lblDisplayB = new System.Windows.Forms.Label();
            this.btnCar1B = new System.Windows.Forms.Button();
            this.btnCar0B = new System.Windows.Forms.Button();
            this.btnOpenB = new System.Windows.Forms.Button();
            this.btnCloseB = new System.Windows.Forms.Button();
            this.lblDoorB = new System.Windows.Forms.Label();
            this.grpB = new System.Windows.Forms.GroupBox();
            this.btnLog = new System.Windows.Forms.Button();
            this.lblLog = new System.Windows.Forms.Label();
            this.lblInfo = new System.Windows.Forms.Label();
            this.grid = new System.Windows.Forms.DataGridView();
            this.toolTip1 = new System.Windows.Forms.ToolTip(this.components);
            ((System.ComponentModel.ISupportInitialize)(this.grid)).BeginInit();
            this.shaftA.SuspendLayout();
            this.shaftB.SuspendLayout();
            this.grpA.SuspendLayout();
            this.grpB.SuspendLayout();
            this.SuspendLayout();
```

```
Form1.Designer.cs | Elevator.cs | Form1.cs | Form1.cs [Design] | LogService.cs | ElevatorLog.cs | Program.cs | Form1.resx
ElevatorApp | ElevatorApp.Form1 | toolTip1

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Windows.Forms;
using Button = System.Windows.Forms.Button;
using Timer = System.Windows.Forms.Timer;

namespace ElevatorApp
{

    public partial class Form1 : Form, IElevatorObserver
    {

        static readonly Color Back = Color.FromArgb(24, 28, 40);

        static readonly Color Card = Color.FromArgb(38, 44, 62);

        static readonly Color Key = Color.FromArgb(52, 60, 84);

        static readonly Color Ink = Color.FromArgb(235, 238, 245);

        static readonly Color Dim = Color.FromArgb(139, 147, 168);

        static readonly Color Teal = Color.FromArgb(46, 196, 182);

        static readonly Color Amber = Color.FromArgb(255, 183, 3);

        6 references
        readonly Dispatcher dispatcher = new Dispatcher();

        readonly Timer clock = new Timer { Interval = 50 };    // फिफ्ट दिलो र स्मूथ हिँड्नका लागि इन्टरभल ५० बनाइएको छ
        3 references
        readonly bool[] hallLit = new bool[2];                 // call button lights, one per floor

        readonly ElevatorContext liftA = new ElevatorContext("A", 0);
        3 references
        readonly ElevatorContext liftB = new ElevatorContext("B", 1);
        2 references
        Button[] hallBtn;
        2 references
        Label[] floorStatus;
        9 references
        LogService logs;

        1 reference
        public Form1()
        {
            InitializeComponent();

            dispatcher.Lifts.Add(liftA);
            dispatcher.Lifts.Add(liftB);

            hallBtn = new Button[] { btnReq0, btnReq1 };
            floorStatus = new Label[] { lblFloorStatus0, lblFloorStatus1 };
            btnReq0.Tag = 0; btnReq1.Tag = 1;

62 %  No issues found  Ln: 181, Ch: 2  SPC  CRLF  UTF-8
```

rm1.Designer.cs | **Elevator.cs** | Form1.cs | Form1.cs [Design] | LogService.cs | ElevatorLog.cs | Program.cs | Form1.resx

ElevatorApp | ElevatorApp.MovingState | Tick(ElevatorContext lift)

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Windows.Forms;

namespace ElevatorApp
{
    // Floors are stored as 0 and 1 inside the program; Show() decides how they look on screen.
    9 references
    public static class Floors
    {
        1 reference
        public const int Offset = 1;     // 1 = screen shows floors 1 and 2, 0 = screen shows floors 0 and 1
        9 references
        public static int Show(int floor) { return floor + Offset; }
    }

    // ---------- Observer pattern ----------
    6 references
    public interface IElevatorObserver
    {
        5 references
        void OnLiftChanged(ElevatorContext lift);
    }

    2 references
    public delegate void LogWriter(string lift, int floor, string action);

    // Observer that writes every real change of a lift to the log (through a delegate)
    2 references
    public class LogObserver : IElevatorObserver
    {
        2 references
        readonly LogWriter write;
        2 references
        readonly Dictionary<string, string> last = new Dictionary<string, string>();

        1 reference
        public LogObserver(LogWriter write) { this.write = write; }

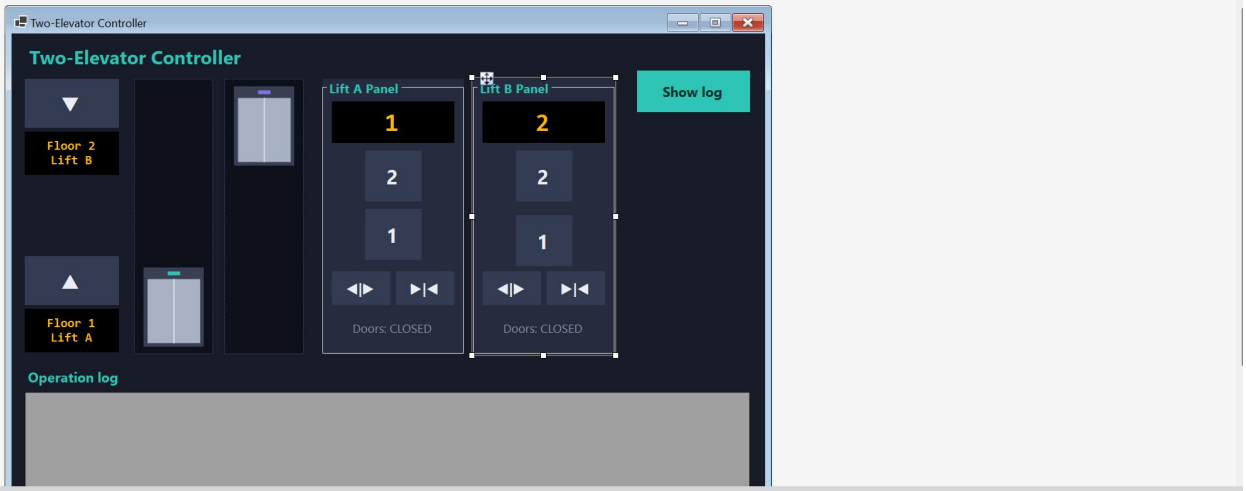
        2 references
        public void OnLiftChanged(ElevatorContext lift)
        {
            string action = lift.IsMoving ? "Moving to floor " + Floors.Show(lift.Target) : lift.State.Name;
            string before;
            if (last.TryGetValue(lift.Name, out before) && before == action) return;
            last[lift.Name] = action;
            write("Lift " + lift.Name, Floors.Show(lift.Floor), action);
        }
    }

    // ---------- State pattern ----------
    5 references
```

No issues found | Ln: 243, Ch: 2 | SPC | CRLF | UTF-8 with BOM

Form1.Designer.cs* | Elevator.cs | Form1.cs* | **Form1.cs [Design]*** | LogService.cs | ElevatorLog.cs | Program.cs | Form1.resx*

Scaling on your main display is set to 150%. Consider opening the WinForms Designer in DPI-Unaware Mode. More info



toolTip1

Form1.Designer.cs* | Elevator.cs | Form1.cs* | Form1.cs [Design]* | **LogService.cs** | ElevatorLog.cs | Program.cs | Form1.resx*

ElevatorApp | ElevatorApp.LogService.Entry | Floor

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;

namespace ElevatorApp
{
    public class LogService
    {
        public class Entry { public string Elevator; public int Floor; public string Action; }

        readonly ElevatorLog db;

        readonly BackgroundWorker saver = new BackgroundWorker();

        readonly BackgroundWorker loader = new BackgroundWorker();

        3 references
        readonly object queueLock = new object();

        readonly Queue<Entry> pending = new Queue<Entry>();

        public bool AutoRefresh;
        3 references
        public event Action<DataTable> Loaded;

        public event Action<Exception> Failed;


        public LogService(ElevatorLog db)
        {
            this.db = db;

            saver.DoWork += (s, e) =>
            {
                while (true)
                {
                    Entry x;
                    lock (queueLock)
                    {
                        if (pending.Count == 0) break;
                        x = pending.Dequeue();
                    }
                    if (x != null) db.Add(x.Floor, x.Action, x.Elevator);
                }
            };

            saver.RunWorkerCompleted += SaverDone;
            loader.DoWork += (s, e) => e.Result = db.Load();
            loader.RunWorkerCompleted += LoaderDone;
        }
```

Form1.Designer.cs* Elevator.cs Form1.cs* Form1.cs [Design]* LogService.cs **ElevatorLog.cs** Program.cs Form1.resx*

ElevatorApp | ElevatorApp.ElevatorLog | Load()

```
using System;
using System.Data;
using System.Data.SQLite;
using System.IO;
using System.Windows.Forms;

namespace ElevatorApp
{
    4 references
    public class ElevatorLog
    {
        4 references
        readonly string connStr;

        1 reference
        public ElevatorLog()
        {
            string folder = Path.Combine(Application.StartupPath, "Data");
            Directory.CreateDirectory(folder);
            connStr = "Data Source=" + Path.Combine(folder, "elevator.db") + ";Version=3;";

            using (var con = new SQLiteConnection(connStr))
            {
                con.Open();
                new SQLiteCommand(
                    "CREATE TABLE IF NOT EXISTS Log (Id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                    "Elevator TEXT, Floor INTEGER, Action TEXT, LoggedAt TEXT)", con).ExecuteNonQuery();
            }
        }

        1 reference
        public DataTable Load()
        {
            var table = new DataTable();
            using (var da = new SQLiteDataAdapter("SELECT * FROM Log ORDER BY Id DESC", connStr))
                da.Fill(table);
            return table;
        }

        1 reference
        public void Add(int floor, string action, string elevator)
        {
            using (var da = new SQLiteDataAdapter("SELECT * FROM Log WHERE 1=0", connStr))
            using (var builder = new SQLiteCommandBuilder(da))
            {
                var table = new DataTable();
                da.Fill(table);

                DataRow row = table.NewRow();
                row["Elevator"] = elevator;
                row["Floor"] = floor;
                row["Action"] = action;
                row["LoggedAt"] = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");
                table.Rows.Add(row);
```

62 % | No issues found | Ln: 55, Ch: 2 | SPC | CRLF | UTF-8 with BOM

ElevatorApp | ElevatorApp.Program | Main()

```
namespace ElevatorApp
{
    0 references
    internal static class Program
    {
        /// <summary>
        ///  The main entry point for the application.
        /// </summary>
        [STAThread]
        0 references
        static void Main()
        {
            // To customize application configuration such as set high DPI settings or default font,
            // see https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new Form1());
        }
    }
}
```